

Simulazione Java: Arena dei Maghi (Free-for-All)

Simulazione di combattimento a turni tra maghi.

Il gioco è di tipo 'tutti contro tutti' (free-for-all): ogni mago combatte contro tutti gli altri fino a quando ne rimane uno solo: il vincitore.

1. Obiettivo della simulazione

L'obiettivo è simulare un combattimento tra più maghi utilizzando regole precise, statistiche e turni. Vince il mago che rimane vivo al termine della simulazione.

2. Gioco tutti contro tutti (Free-for-All)

Nel free-for-all tutti i maghi sono avversari tra loro. Non esistono squadre.

L'ordine dei turni è dato dalla velocità del mago (ordine decrescente).

Ad ogni turno:

- vengono considerati solo i maghi ancora vivi
- i maghi agiscono uno alla volta
- ogni mago sceglie cosa fare nel proprio turno

3. Valori (statistiche) del mago

Ogni mago è caratterizzato dai seguenti valori:

- **HP** (punti vita): indicano la salute del mago. Quando arrivano a 0, il mago è sconfitto.
- **Mana**: energia magica necessaria per lanciare incantesimi.
- **Potenza magica**: aumenta il danno degli incantesimi offensivi.
- **Difesa**: riduce il danno ricevuto.
- **Velocità**: determina l'ordine dei turni.

4. Assegnazione dei valori iniziali (Random controllato)

I valori iniziali dei maghi vengono assegnati in modo casuale (random), ma entro intervalli controllati per garantire equilibrio.

Esempio di intervalli:

- HP: 40 – 60
- Mana: 20 – 40
- Potenza magica: 5 – 10
- Difesa: 3 – 8
- Velocità: 1 – 10

Questo metodo rende ogni partita diversa senza creare maghi troppo forti o troppo deboli.

5. Azioni possibili nel turno

Quando è il turno di un mago, può scegliere una delle seguenti azioni:

1) Attaccare un altro mago

- consuma mana
- riduce gli HP del mago bersaglio

2) Curarsi

- consuma mana
- aumenta i propri HP (senza superare il massimo)

3) Riposo (opzionale)

- non attacca
- recupera una piccola quantità di mana

6. Calcolo del danno

Quando un mago attacca, il danno viene calcolato con la seguente formula:

$\text{danno} = \text{danno base dell'incantesimo} + \text{potenza magica dell'attaccante} - \text{difesa del bersaglio}$

Se il danno risulta minore di 1, viene comunque applicato 1 punto di danno.

7. Esempio di simulazione

Esempio:

Merlino:

- HP = 50
- Mana = 30
- Potenza = 8

Morgana:

- HP = 35
- Difesa = 5

Merlino lancia un incantesimo con danno base 10 e costo 7 mana.

Calcolo del danno:

$$10 + 8 - 5 = 13$$

Aggiornamento valori:

- Mana di Merlino: 30 → 23
- HP di Morgana: 35 → 22

8. Fine della simulazione

La simulazione termina quando rimane un solo mago con HP maggiori di 0.
Quel mago è dichiarato vincitore.

9. Classi necessarie (descrizione)

Per realizzare la simulazione in Java in modo ordinato (OOP), è consigliato dividere il programma in più classi, ognuna con una responsabilità precisa. Di seguito sono elencate le classi principali con ciò che devono contenere e fare.

9.1 Classe Wizard (Mago)

Rappresenta un combattente nell'arena.

Serve per:

- memorizzare lo stato del mago (HP, mana, statistiche)
- eseguire azioni come attaccare o curarsi
- aggiornare i propri valori quando subisce danni o spende mana

Attributi tipici:

- String nome
- String alias;
- int hp, hpMax
- int mana, manaMax
- int potenzaMagica
- int difesa
- int velocita
- List<Spell> spellBook

Lo spellBook non è strettamente necessario se tutti i maghi possono usare gli stessi incantesimi. E' consigliato perché permette di differenziare i maghi

Metodi tipici:

- boolean isAlive()
Controlla se il mago è ancora in vita. Ritorna true se gli HP sono maggiori di 0
- void takeDamage(int danno)
Quando un mago lo attacca sottrae il danno dagli HP e impedisce che gli HP vadano sotto 0
- void heal(int quantita)
aggiunge la quantità agli HP quando il mago usa un incantesimo di cura, non supera mai hpMax
- boolean canCast(Spell s)
Controlla se il mago ha abbastanza mana per lanciare un incantesimo confrontando il mana attuale con il costo dell'incantesimo
- void cast(Spell s, Wizard target)
Esegue il lancio di un incantesimo.

- 1 controlla se il mago è vivo
- 2 controlla se può lanciare l'incantesimo
- 3 riduce il mana
- 4 applica l'effetto:
 - ATTACCO → danno al bersaglio
 - CURA → cura se stesso

- void rest()

Permette al mago di riposare nel turno, aumenta il mana di +5 senza superare manaMax

- void regenMana(int quantita) (se previsto)

Recupera una quantità specifica di mana. (pozioni, effetti speciali, rigenerazione automatica)

9.2 Classe Spell (Incantesimo)

Rappresenta un incantesimo lanciabile da un mago.

Serve per:

- descrivere costo e effetto dell'incantesimo
- fornire i dati necessari per calcolare danno o cura

Attributi tipici:

- String nome
- int costoMana
- int valoreBase (danno o cura)
- String tipo (ATTACCO, CURA) *Serve per decidere cosa succede quando si lancia lo spell (toglie HP al mago colpito, aggiunge HP al mago che sceglie di curarsi.).*

Metodi tipici:

- int getCostoMana()
- String getTipo()
- int getValoreBase()

9.3 Classe Arena o Battle (Gestione combattimento)

Gestisce la simulazione: turni, ordine dei maghi, fine partita.

Serve per:

- mantenere la lista dei maghi
- eseguire i turni finché non resta un solo mago vivo
- ordinare i maghi per velocità ad ogni turno
- scegliere/gestire i bersagli (umano o AI)

Attributi tipici:

- List<Wizard> wizards
- Random random
- int turno

Metodi tipici:

- void playMatch() (ciclo fino alla vittoria)
- void playTurn() (gestisce un singolo turno completo)
- List<Wizard> getAliveWizards()
- Wizard getWinner()

9.4 Classe AIController (Decisioni dell'AI) – opzionale

Gestisce le decisioni di un mago controllato dal computer.

Serve per:

- scegliere l'azione nel proprio turno (attacco/cura/riposo)
- scegliere quale incantesimo usare
- scegliere il bersaglio

Esempio di regole semplici:

- se HP < 30% e ho mana: cura
- altrimenti se ho mana: attacco con incantesimo più forte
- altrimenti: riposo o attacco base

9.5 Classe Game o Main (Avvio e input utente)

Punto di ingresso del programma (main).

Serve per:

- creare i maghi (con valori random controllati)
- inizializzare gli incantesimi
- avviare l'Arena/Battle
- (se c'è un giocatore umano) gestire input da console/menu